HTTPS协议

1.HTTPS流程：

对称加密：

```
1.加密和解密所用的密钥相同!
2.效率高、算法公开
```

例子：

> 假设我们要传递一个值:int a = 10;此时客户带持有一个加密密钥：int b =20;服务器也有一个相同的密钥：int b = 20;客户端发送数据加密后为int tmp = a^b;
>
> 服务器接收到数据后-->int recv = tmp; 讲recv ^ b = a^b^b = a^0 =a；就拿到了真正要传递的值！

非对称加密：

> 存在公有密钥和私有密钥；
>
> 两种方式：先用公钥加密，则只能由私钥解密--意味着能解密的人少！！
>
> 先用私钥加密，再用公钥解密--->意味着只要有公钥就能解密！！

数据摘要和数据指纹

数据摘要：

```
第一种应用场景：云盘
客户端上传文件/数据 -->hash散列算法--->形成数据摘要(hash值)--->服务端验证摘要是否曾经在服务端内部存在，如果存在则证明上传的文件内容相同!!!
场景二：
加密用户传递的密码--数据库--hash验证摘要
```

四种方案：

1.双方都使用对称加密：

```
存在很严重的问题：
1.如果是client--->server发送信息，client要发送的信息应该是 info(原始数据) + 密钥K
server收到的是加密过的信息，但是server同样会使用密钥K去对该信息解密！问题就在这里：密钥K在交换时是可以被中间人获取的，那么由于server同样使用密钥K解密，意味着只要是持有密钥K的角色都能对client的数据进行解密，这是绝对不允许的！！
```

2.一方使用非对称加密：

```
client使用公钥进行加密，即message = 原始数据 + 公钥K --->server
数据传递的过程中任何人都可以获取到公钥，server收到加密的信息后，使用自己的私钥M去解密，在clinet-->server的传输方向来看虽然是安全的，但是！！server的响应呢？？server采用私钥M加密rsp，发回client，client采用公钥K解密，这就意味着server的rsp是不安全的！！
```

3.双方都使用非对称加密

```
双方都是用公钥加密信息，自己的私钥解密信息：
双方正式通信之前，先进行密钥交换
client:公钥A，私钥B
server:公钥C，私钥D

1.先交换各自的公钥，client--->公钥A-->server拿到公钥A；server--->公钥C-->client拿到公钥C
2，开始使用对方的公钥加密自己要发送的信息，对方收到使用自己的私钥机密即可！！

但是！也是不安全的！！！
公钥交换阶段时，万一公钥被篡改了呢？？？是不是就不安全了啊！！！
并且非对称加密速度慢，计算开销大
```

4.双方使用对称机密 + 非对称加密

```
首先client啥都不做，server生成一个公钥S和一个私钥S^
1.密钥交换阶段，server向client发送自己的公钥S，此时这个公钥S任何人都可以获取到！包括中间人
2.client拿到服务器的公钥S后，在自己的本地生成一对对称密钥C
3.client如何将自己的对称密钥C发给server呢？？答案是用server的公钥S加密对称密钥C！！密钥也是数据啊！！！加密后发起http请求，server收到请求，使用只有自己知道的私钥S^进行解密，获得密钥C，此时，server和client都知道了密钥C，所以之后的通信都采用对称加密的方式就行了！！

问题来了”如果在公钥S的发送阶段时，中间人直接篡改了公钥S换成自己的公钥R，发送给client，此时client事先并不知道真正的server的公钥的！！当client收到中间人的公钥R时，万一直接用公钥R进行加密自己的对称密钥嗯？？是不是在client看来，此时的server实际上已经是中间人了！！往后所有的通信都会被中间人获得并解密！！
```

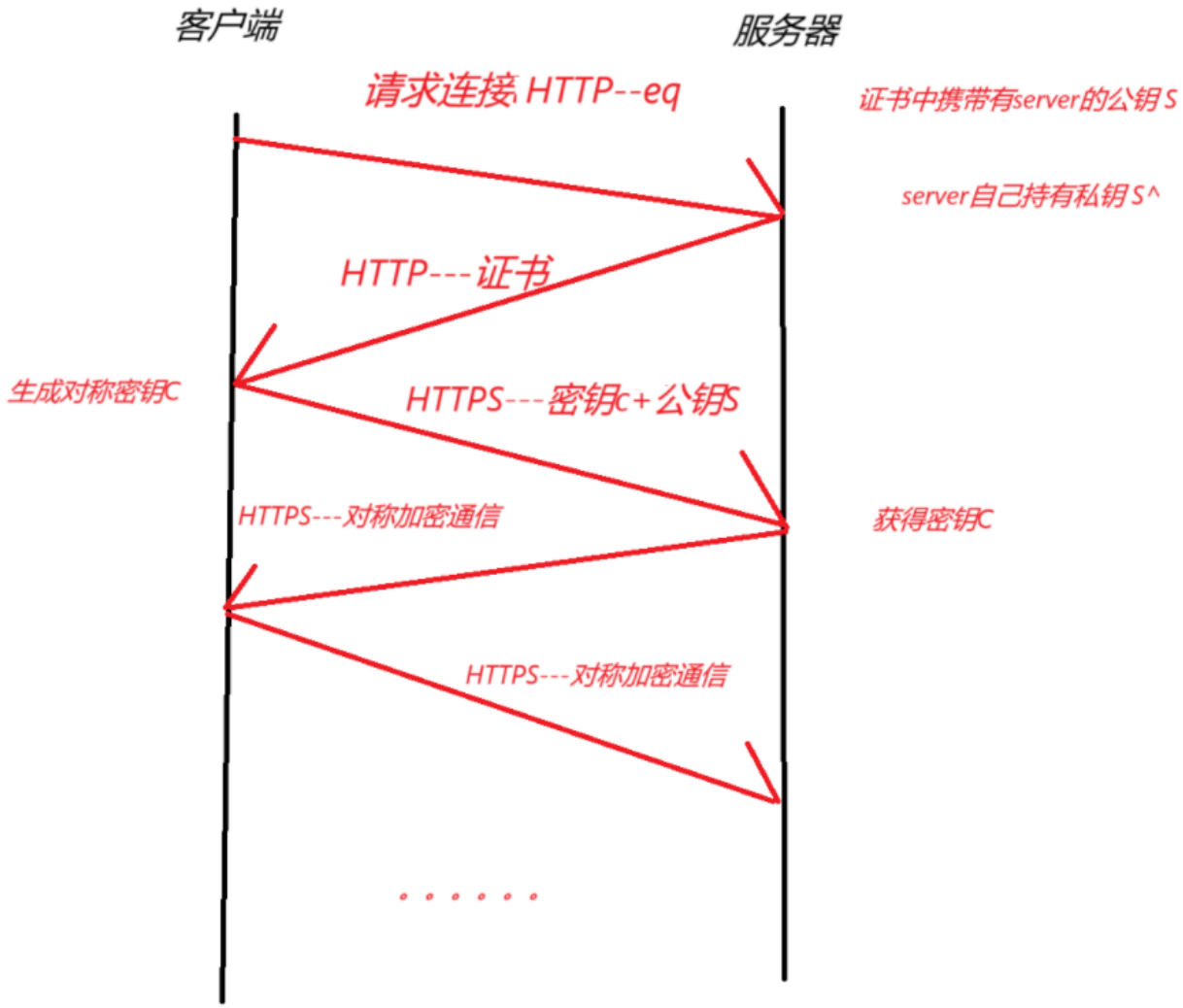
CA证书和签名：

```
数字签名：CA机构向原始数据进行哈希散列形成散列值，并用自己的私钥进行加密-->即这就是签名过程
CA证书：签名 + 证书的明文信息（info）
在client<--->server过程中，首次通信server会先向client发送证书，证书中含有server自己的公钥和证书有效期、CA机构、数字签名等，client(浏览器)通常会内置信任的CA机构的公钥列表，如果client收到证书后对比发现不在自身的信任列表中，则会拒绝或者提示本次连接危险/不安全！！如果在信任列表中，则拿出相应CA机构的公钥进行解密，client首先对比数字签名，对证书的明文信息采用和签发给server证书的CA机构相同的哈希算法得到一个散列值，将其与解密后的签名进行对比，一致则表示信息未被篡改过。

这样即使证书被劫持，中间人在没有CA机构私钥的情况下是无法伪造数据签名的！！！只有持有私钥的人才能签发证书！！

即第五种安全方案！
但是！！！也有安全问题，如果中间人自己持有一份真正的CA机构颁发的合法证书，在client和server的证书交换阶段直接将server发送的证书全部替换为自己的证书，发给client呢？
别忘记了！！证书中的info中含有域名信息！！！！！client会进行域名验证的！！
```

HTTPS整体流程：


客户端
服务器
请求连接 HTTP--eq
证书中携带有server的公钥S
server自己持有私钥S^
HTTP---证书
生成对称密钥C
HTTPS---密钥c+公钥S
HTTPS---对称加密通信
获得密钥C
HTTPS---对称加密通信
.....